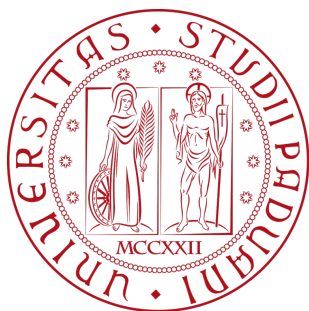




Università degli Studi di Padova

Corso di Laurea in Informatica



VitalPath

Sistema di prenotazione prelievi online

Relazione di Progetto — A.A. 2025-2026

Indirizzo del sito:

tecweb.studenti.math.unipd.it/amenegal

Mail Referente:

andrea.menegaldo@studenti.unipd.it

Credenziali di Accesso per il Test:

Ruolo	Username / Codice Fiscale	Password
Utente	user	user
Amministratore	admin	admin
Utente	RSSMRA80A01H501X	user
Utente	BNCANN85B45F205Y	user

Gruppo Componenti:

Nome e Cognome	Matricola
----------------	-----------

Roberto Doroftei	2111031
Francesco Marcon	2110990
Andrea Menegaldo	2116426
Edis Hodja	2116422

Indice

1	Introduzione	4
2	Analisi dei requisiti	4
2.1	Target di utenza e obiettivi di business	4
2.2	Requisiti di funzionamento	5
3	Organizzazione del lavoro e progettazione	5
3.1	Approccio metodologico e suddivisione dei ruoli	5
3.1.1	Dettaglio delle attività individuali	5
3.2	Gestione dei file e della directory	6
3.3	Architettura dell'ambiente di sviluppo (Docker e Makefile)	7
4	Implementazioni lato front-end	7
4.1	Struttura delle sezioni e funzionalità per ruolo	7
4.2	Mappa di navigazione del sito	8
4.3	Validazione dei dati e sicurezza	8
4.4	Operazioni dedicate all'amministratore	9
4.5	Implementazione del CSS	9
5	Accessibilità e usabilità	10
5.1	Studio dei colortest e conformità WCAG 2.1	10
5.2	Colori utilizzati e palette del design system	10
5.3	Implementazioni per la navigazione da tastiera e tecnologie assistive	11
6	Implementazioni lato back-end	12
6.1	Architettura dati e interazione tramite PDO	12
6.2	Gestione delle sessioni, inserimento e cancellazione sicura	13
7	Motivazione delle scelte e parti critiche	13
7.1	Sviluppo in PHP puro senza framework ausiliari	13
7.2	Utilizzo del codice fiscale come identificatore univoco logico	13
7.3	Punti critici e limitazioni dell'architettura attuale	14
8	Test	14
8.1	Validazione del markup	14
8.2	Estensioni Chrome per l'analisi di accessibilità	14
8.3	Audit con Lighthouse	15
9	Note conclusive	15

1 Introduzione

Il progetto VitalPath nasce dall'esigenza di realizzare una piattaforma web dedicata alla gestione e alla prenotazione degli appuntamenti per i prelievi del sangue all'interno di una struttura sanitaria o di un laboratorio d'analisi locale. Nel panorama attuale, l'interazione tra il cittadino e le strutture sanitarie per questo tipo di servizi è spesso delegata a centralini telefonici o a portali complessi, che presentano grossi problemi in termini di usabilità ed accessibilità.

L'applicazione proposta vuole abbattere queste criticità offrendo un'interfaccia efficace. Da un lato, fornisce ai pazienti un ambiente privo di frizioni informatiche per monitorare lo stato del proprio profilo sanitario e riservare un singolo slot temporale per l'esame. Dall'altro lato mette a disposizione del personale amministrativo un pannello di controllo globale per supervisionare il flusso delle richieste, verificare le statistiche del centro e intervenire con azioni di annullamento o revoca in caso di necessità.

L'intero impianto applicativo è stato sviluppato rifiutando l'adozione di framework preimpostati, focalizzandosi sull'utilizzo nativo delle tecnologie web standard. Questa scelta ci ha permesso di esercitare un controllo più preciso sulla pulizia del codice garantendo la totale separazione tra la struttura dei contenuti, lo stile della presentazione e le logiche di comportamento lato client e server. Un aspetto importante è stato quello del rispetto dei requisiti di accessibilità e inclusività, assicurando che la piattaforma risulti pienamente navigabile anche tramite tecnologie assistive o dispositivi a schermo ridotto.

2 Analisi dei requisiti

2.1 Target di utenza e obiettivi di business

La piattaforma che abbiamo realizzato è progettata per interfacciarsi con due macro-categorie di utenti, ciascuna caratterizzata da necessità differenti:

- **Utente(User):** Rappresenta il soggetto che necessita del servizio. L'obiettivo principale è minimizzare il tempo speso per la registrazione o il login e per la prenotazione dell'esame. Garantire una consultazione immediata dei propri dati e appuntamenti. L'interfaccia deve accogliere con la stessa efficacia sia utenti famigliari con le tecnolgie sia fasce di popolazione meno avanzate con gli strumenti informatici (utenti anziani), eliminando ambiguità testuali o flussi inutili.
- **Amministratore(Admin):** Necessita di una panoramica immediata e centralizzata del carico di lavoro. L'obiettivo è supervisionare le prenotazioni attive per prevenire il sovrappollamento della struttura.

2.2 Requisiti di funzionamento

L'analisi delle specifiche ha portato alla definizione di regole ben definite, implementate per garantire l'integrità dei flussi di lavoro:

1. **Vincolo del singolo appuntamento:** Al fine di evitare coincidenze con gli slot temporali, per evitare un attacco DOS (denial of service) e garantire la disponibilità del servizio a tutta l'utenza, un paziente registrato può mantenere un solo appuntamento attivo nello stato «prenotato». Il sistema vieta l'accesso all'area di prenotazione finché l'appuntamento non viene effettuato o cancellato.
2. **Stati della prenotazione:** Il modello dati implementato prevede tre stati principali per una prenotazione: **prenotato**, **effettuato** e **cancellato**. Quando si prenota un esame il suo stato è **prenotato** di default.
3. **Finestra temporale di prenotazione:** Il sistema genera gli slot disponibili esclusivamente per una finestra di sei giorni lavorativi consecutivi a partire dal giorno successivo a quello corrente (dal lunedì al sabato, dalle 08:00 alle 12:30).
4. **Ripristino della possibilità di prenotazione:** Nel momento in cui un amministratore cancella un appuntamento già fissato da un paziente, l'utente riacquisisce la possibilità di inserire una nuova prenotazione nel sistema.

3 Organizzazione del lavoro e progettazione

3.1 Approccio metodologico e suddivisione dei ruoli

Lo sviluppo della web app ha seguito un approccio incrementale basato su continui cicli di integrazione e revisione del codice. La collaborazione del gruppo si è strutturata attraverso l'utilizzo di un repository GitHub e sessioni di allineamento costanti sulla piattaforma vocale Discord, per le comunicazioni semplici si è operato per un gruppo WhatsApp, consentendo una comprensione maggiore dell'intera applicazione web riunendo le attività individuali.

3.1.1 Dettaglio delle attività individuali

Per garantire un avanzamento parallelo e sincronizzato del progetto, le diverse sezioni di lavoro sono state suddivise cercando di ottenere un equilibrio tra i carichi di lavoro di backend, frontend e infrastruttura.

- **Roberto Doroftei:** Ha ingegnerizzato l'intero ecosistema di virtualizzazione usando Docker, configurando i servizi di networking per Apache/PHP, MariaDB e l'interfaccia di amministrazione. Ha centralizzato la gestione dei flussi automatizzando le istruzioni del terminale tramite un Makefile per il team.
- **Francesco Marcon:** Ha progettato la logica applicativa core in PHP nativo implementando i controlli di sicurezza sia per la fase di login che di registrazione degli utenti. Ha inoltre coordinato il workflow dei rami di sviluppo e la risoluzione dei conflitti sul repository.

- **Andrea Menegaldo:** Si è occupato della modellazione concettuale e logica del database, strutturando le tabelle in forma normale e definendo i vincoli di integrità referenziale. Sul fronte del frontend, ha architettato il layout responsivo del sito e il sistema globale di impaginazione e variabili in CSS puro. Ha inoltre eseguito i test di contrasto cromatico e usabilità visiva.
- **Edis Hodja:** Si è occupato dell'interfaccia e dei comportamenti dell'area amministratore, integrando le relative metriche e funzioni di cancellazione degli slot. Ha implementato gli script JavaScript per i controlli interattivi e reattivi del client.

3.2 Gestione dei file e della directory

La cartella dei file evidenzia una separazione distinta tra i componenti di configurazione dell'ambiente di virtualizzazione locale Docker, gli script SQL di inizializzazione della persistenza e la cartella sorgente dell'applicazione web, eliminando accoppiamenti pesanti tra codice e infrastruttura:

```
progetto_tec_web/
├─ docker-compose.yml
├─ Makefile
├─ README.md
├─ db/
│   └─ init_scripts/
│       ├── 0_init_db.sql
│       └─ 1_grant_permissions.sql
├─ src/
│   ├── admin_dashboard.js
│   ├── admin_dashboard.php
│   ├── cancella_prenotazione.php
│   ├── dashboard.js
│   ├── dashboard.php
│   ├── db.php
│   ├── favicon.svg
│   ├── index.php
│   ├── login.js
│   ├── login.php
│   ├── logo.php
│   ├── logout.php
│   ├── prenotazioni.js
│   ├── prenotazioni.php
│   ├── registrazione.js
│   ├── registrazione.php
│   └─ style.css
└─ web/
    └─ Dockerfile
```

3.3 Architettura dell'ambiente di sviluppo (Docker e Makefile)

Per evitare il riscontro di anomalie software dovute dalle differenze tra i sistemi operativi dei componenti del gruppo, l'intero stack è stato virtualizzato tramite Docker Compose in tre container isolati nella medesima rete virtuale:

- `web`: Espone un server HTTP Apache basato su PHP 8.2 sulla porta locale 8000.
- `database`: Definisce un motore relazionale MariaDB con script SQL.
- `phpmyadmin`: Un'interfaccia grafica esposta sulla porta 8080.

L'interazione con l'ambiente è semplificata da un `Makefile` tramite il singolo comando `make rebuild`. Tutte le informazioni per accedere al sito sono contenute nel file `README.md`.

4 Implementazioni lato front-end

4.1 Struttura delle sezioni e funzionalità per ruolo

L'interfaccia utente è stata progettata seguendo una gerarchia, adattando dinamicamente i componenti visibili in base all'autenticazione presente nella sessione dell'utente che visita il sito.

- **Visitatore:** Accedendo alla homepage si ritrova di fronte ad una panoramica informativa sui servizi del centro, sulle modalità di preparazione agli esami e sugli orari della struttura. La barra di navigazione mostra i collegamenti alla finestra di autenticazione e registrazione.
- **Utente registrato:** Una volta autenticato, il menu mostra anche il collegamento all'area personale e se non è presente alcuna prenotazione attiva del link per accedere alla finestra di prenotazione.
- **Amministratore:** Il login con credenziali amministrative reindirizza al pannello di controllo dell'intero sistema. Questa sezione non presenta le funzioni di prenotazione ma porta ad una schermata di monitoraggio avanzata contenente le statistiche del centro e l'elenco di tutte le visite associate al rispettivo utenti registrati.

4.2 Mappa di navigazione del sito

Di seguito è riportata la mappa di navigazione del sito, pagina per pagina. Per le pagine con un form (login e registrazione) sono distinti i collegamenti di navigazione statici dall'esito dell'invio del form.

```
Home (index.php)                                [non autenticato]
  |--> Login (login.php)
  |--> Registrazione (registrazione.php)

Registrazione (registrazione.php)
  |--> Login (login.php)
  |--> Home (index.php)
  ==> Dashboard (dashboard.php)                [esito: registrazione riuscita]

Login (login.php)
  |--> Registrazione (registrazione.php)
  |--> Home (index.php)
  ==> Dashboard (dashboard.php)                [esito: utente]
  ==> Admin Dashboard (admin_dashboard.php)    [esito: admin]

Dashboard (dashboard.php)                        [utente autenticato]
  |--> Home (index.php)
  |--> Prenotazioni (prenotazioni.php)         [solo se nessuna prenotazione attiva]
  |--> Logout -----> Home (index.php)

Admin Dashboard (admin_dashboard.php)           [admin]
  |--> Home (index.php)
  |--> Logout -----> Home (index.php)

Prenotazioni (prenotazioni.php)                 [utente autenticato, non admin, senza
prenotazioni attive]
  |--> Home (index.php)
  |--> Dashboard (dashboard.php)
```

Legenda: `-->` indica un collegamento di navigazione presente nella pagina (link o pulsante), `==>` indica il reindirizzamento a seguito dell'invio riuscito di un form. Le pagine di errore non sono rappresentate.

4.3 Validazione dei dati e sicurezza

La sicurezza e l'integrità dei dati inseriti sono garantite sul client tramite script dedicati in JavaScript standard puro prevenendo l'inoltro di richieste errate o incomplete verso il server.

Nella pagina di registrazione e di modifica dei dati, dietro alla compilazione del form per verificare la conformità dei campi compilati dall'utente è presente una logica con delle limitazioni:

- L'indirizzo email deve corrispondere al pattern validato.

- Il codice fiscale deve corrispondere al pattern validato (eccetto per admin e user) e viene forzato in lettere maiuscole.
- Il numero di telefono viene bloccato a una lunghezza fissa di 10 cifre numeriche.
- La password viene controllata in termini di robustezza basata sulla lunghezza della stringa rifiutando password con meno di 4 caratteri.

In caso di anomalie, l'invio del form viene interrotto tramite un metodo `e.preventDefault()` che attiva una funzione che mostra un avviso in cima al form.

4.4 Operazioni dedicate all'amministratore

Per ottimizzare l'efficienza delle operazioni dell'amministratore, lo script `admin_dashboard.js` presenta un sistema di filtraggio dei record della tabella in tempo reale senza richiedere il caricamento della pagina. Agganciando un `EventListener` all'input di ricerca viene analizzata la stringa digitata, confrontandola con il contenuto testuale di ogni riga della tabella.

Inoltre, è stato integrato un algoritmo di ordinamento per colonne tramite l'elemento `.sort()` degli array JavaScript. Cliccando sulle intestazioni abilitate le righe vengono ordinate alfabeticamente o numericamente a seconda del tipo di dato (es. identificativo numerico, ordine alfabetico o per data), modificando l'attributo di accessibilità `aria-sort` tra i valori `ascending` e `descending`.

4.5 Implementazione del CSS

Il foglio di stile del progetto è stato realizzato utilizzando CSS nativo, organizzando le regole in modo modulare e facendo uso di custom properties per centralizzare colori, tipografia, spaziature e altre caratteristiche grafiche comuni. Sono state inoltre definite alcune utility classes riutilizzabili per la gestione delle spaziature e dei layout Flexbox, riducendo la duplicazione delle regole e mantenendo uniforme la struttura delle pagine. Particolare attenzione è stata dedicata al comportamento responsive. Attraverso diverse media query, il layout viene adattato alle dimensioni dello schermo, modificando la disposizione degli elementi, il numero di colonne delle griglie e la struttura di alcuni componenti. Per schermi inferiori a 960 px, ad esempio, le tabelle vengono trasformate in blocchi verticali, evitando lo scorrimento orizzontale e mantenendo leggibili i dati. Infine, è stata realizzata una modalità specifica per la stampa mediante media print, nella quale vengono nascosti gli elementi non necessari, come intestazione e footer, eliminate ombre ed effetti grafici e impostati colori e caratteri più adatti alla stampa.

5 Accessibilità e usabilità

5.1 Studio dei colortest e conformità WCAG 2.1

La parte di frontend della piattaforma abbiamo cercato di farla rispettando i requisiti stabiliti dalle linee guida WCAG 2.1 al livello AA. Le scelte cromatiche, definite all'interno del file `style.css`, non sono solo una scelta estetica, ma sono state verificate con strumenti di colortest per assicurare un contrasto di luminanza adeguato. I Test eseguiti ci hanno permesso di selezionare un pantone adeguato.

Il testo principale adotta il colore nero su sfondo bianco puro sviluppando un rapporto di contrasto pari a 16.7:1, ampiamente superiore alla soglia minima di 4.5:1 richiesta per il testo normale. I messaggi di errore usano un background chiaro abbinato a un testo rosso scuro, garantendo leggibilità del testo e la chiara evidenziazione dell'allert.

5.2 Colori utilizzati e palette del design system

Per rendere riconoscibile la piattaforma abbiamo selezionato una palette che richiama, sul piano percettivo e su quello culturale, l'ambito sanitario in cui il servizio si colloca.

Il colore primario è il blu (`#0066cc`), applicato su sfondo bianco (`#f8fafc`). La combinazione non è casuale. Nella percezione comune, occidentale e italiana in particolare, il blu è associato a fiducia, competenza e stabilità, per questo banche, assicurazioni e strutture sanitarie lo adottano diffusamente nella propria identità visiva, spesso a preferenza del rosso o del verde acceso, colori più energici ma anche più ansiogeni in un contesto clinico. Trattandosi di un servizio di prenotazione per prelievi del sangue, chi accede al sito può già trovarsi in uno stato di lieve apprensione legato all'esame stesso. La scelta del blu risponde proprio a questa condizione, trasmettendo calma e serietà senza risultare né freddo né allarmante.

Il bianco, usato come sfondo dominante, richiama a sua volta l'immaginario degli ambienti clinici, camici, pareti, strumentazione, culturalmente associato a pulizia, igiene e sterilità. Oltre al valore simbolico, garantisce anche il massimo contrasto di luminanza possibile con il testo scuro sovrapposto, un aspetto non secondario per un'utenza che può includere persone anziane o con ridotta capacità visiva, come indicato nei requisiti di accessibilità del progetto.

Il verde, il giallo e il rosso, riservati rispettivamente allo stato di successo, avviso e allo stato di errore, seguono una convenzione semantica ormai consolidata nell'interfaccia utente occidentale. Per questo sono usati con parsimonia, solo dove serve un segnale immediato, per non sovraccaricare visivamente un'interfaccia pensata, prima di tutto, per rassicurare chi la usa.

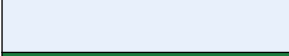
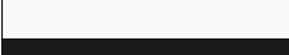
Colore	Codice esadecimale	Ruolo semantico nel sito
	<code>#0066cc</code>	Color Primary: Bottoni, elementi attivi
	<code>#004d99</code>	Color Primary darck: hover dei bottoni
	<code>#e8f0fb</code>	Sfondi focus ring, avvisi info
	<code>#1b7a3e</code>	Notifiche positive di conferme
	<code>#fff8e1</code>	Box preparazione esami e avvisi
	<code>#d32f2f</code>	Allerta form, azioni distruttive
	<code>#f8fafc</code>	Sfondo applicativo delle pagine
	<code>#1a1a1a</code>	Testo principale

Tabella 1: Palette cromatica reale estratta dai fogli di stile

5.3 Implementazioni per la navigazione da tastiera e tecnologie assistive

Per consentire l'utilizzo autonomo della piattaforma a utenti con disabilità motorie o visive, è stata implementata una serie di accorgimenti tecnici nel codice HTML5 e CSS3:

- **Skip Links:** In cima a ogni documento è inserito un collegamento di salto strutturale (`<a href="#main-content" class="skip-link">`). Questo elemento rimane invisibile a schermo durante la navigazione standard con il mouse, ma si palesa graficamente nell'angolo superiore non appena rileva il focus da tastiera (tasto `TAB`), permettendo di superare i blocchi ripetitivi del menu superiore e di atterrare direttamente sul nucleo informativo della pagina.
- **Gestione del Focus Ring:** È stata interdetta la rimozione indiscriminata degli indicatori di focus. Nel foglio di stile è stato ingegnerizzato un doppio anello di focus tramite la pseudo-classe `:focus-visible`, che applica un outline marcato blu (`#0066cc`) distanziato dall'elemento tramite un ombreggiatura interna bianca, assicurando visibilità su qualunque sfondo senza penalizzare l'estetica per gli utenti che utilizzano il mouse.
- **Elementi interattivi accessibili:** All'interno del modulo di prenotazione (`prenotazioni.php`), i controlli di selezione del giorno e dell'orario, originariamente mappati come radio button nativi (nascosti graficamente per ragioni di layout), sono stati resi pienamente operativi per la tastiera. Ai relativi tag label è stato applicato l'attributo `tabindex="0"` per l'inclusione nel flusso di tabulazione, integrando gli attributi semantici `role="radio"` e `aria-checked`. Tramite JavaScript, è stata mappata la navigazione nativa con le frecce direzionali (`ArrowRight`, `ArrowLeft`) per scorrere ciclicamente all'interno degli slot disponibili, replicando fedelmente il comportamento dei componenti d'interfaccia predefiniti dei sistemi operativi.

Non è stato ritenuto utile l'implementazione di `stamp`, in quanto riteniamo che il nostro sito sia già facilmente navigabile.

6 Implementazioni lato back-end

6.1 Architettura dati e interazione tramite PDO

Il motore di persistenza si appoggia su una base di dati relazionale MariaDB, strutturata in forma normale per eliminare la ridondanza informativa. Il modello si articola attorno alle due entità principali **persona** e **prenotazione**, legate da un vincolo di integrità referenziale uno-a-molti (**persona_id** -> **cf** con clausola **ON DELETE CASCADE**).

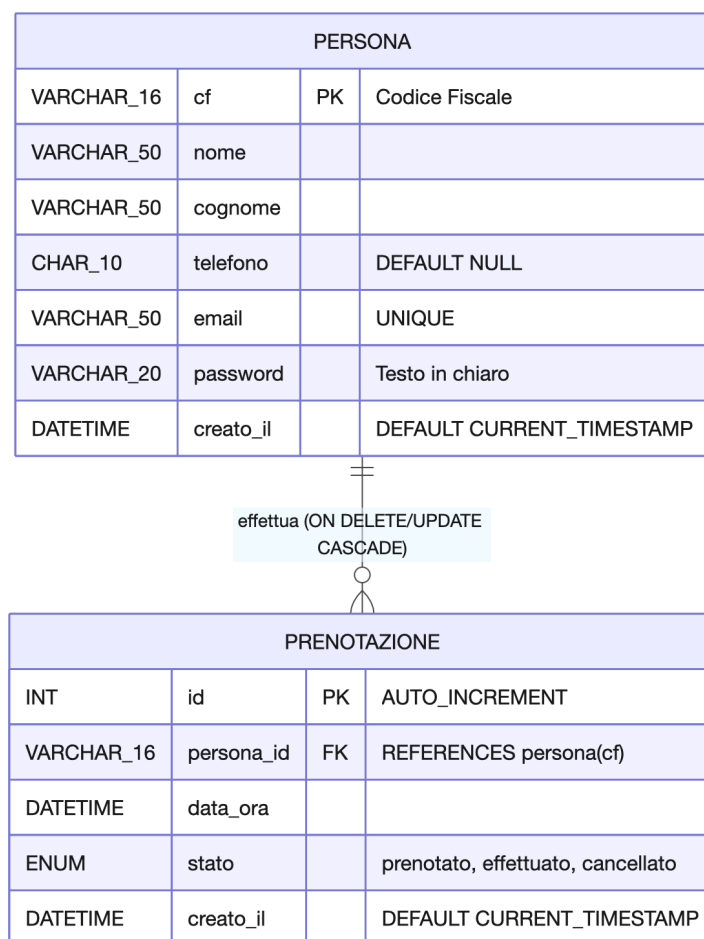


Figura 1: Modello logico/relazionale del database

L'intera logica di interazione con la base di dati è centralizzata nel modulo **db.php**, il quale istanzia un oggetto globale appartenente all'estensione **PDO** (PHP Data Objects). Per inibire sistematicamente attacchi di tipo SQL Injection nei form applicativi, tutte le query che accettano parametri inseriti dall'utente sfruttano i **prepared statement** con parametri posizionali o nominali. Un esempio significativo è riscontrabile nella logica di login, dove la query viene preparata con un segnaposto nominale o posizionale prima di legarvi in modo sicuro le variabili provenienti dagli input di testo.

6.2 Gestione delle sessioni, inserimento e cancellazione sicura

Lo stato di autenticazione dell'utente è preservato attraverso l'uso delle sessioni native di PHP, inizializzate tramite la chiamata `session_start()`. Per mitigare le vulnerabilità di Session Fixation, la procedura esegue un `session_regenerate_id(true)` non appena l'utente supera la verifica delle credenziali nel file `login.php`.

Il sistema implementa logiche server-side distinte per le operazioni di manipolazione dei record:

- **Inserimento (Prenotazione slot):** All'interno del file `prenotazioni.php`, prima di eseguire la query di inserimento, viene verificato lato server che lo slot cronologico scelto dall'utente non sia già occupato. La validazione è controllata tramite una tabella di slot attivi e una verifica preventiva sull'orario scelto.
- **Cancellazione sicura (Paziente):** La revoca di un appuntamento da parte dell'utente è gestita dallo script `cancella_prenotazione.php`. In questa versione del progetto l'azione viene attivata con un parametro `id` passato tramite GET, e la query aggiorna lo stato solo se la prenotazione appartiene alla sessione corrente e se è ancora nello stato `prenotato`.
- **Cancellazione e revoca (Amministratore):** Nel file `admin_dashboard.php`, l'amministratore può annullare un appuntamento attivo inviando una richiesta POST con l'ID della prenotazione. Il controllo di autorizzazione è basato sul codice fiscale in sessione, confrontato con la costante di amministrazione. In caso di accesso non autorizzato, il codice reindirizza verso `login.php`.

7 Motivazione delle scelte e parti critiche

7.1 Sviluppo in PHP puro senza framework ausiliari

La scelta di non adottare framework moderni (quali Laravel o Symfony) è stata guidata dalla volontà di comprendere a fondo le dinamiche fondamentali del protocollo HTTP e della gestione dello stato sul web. L'implementazione di un sistema di templating rudimentale basato sulla funzione nativa `str_replace()` per i menu e le inclusioni dei file di intestazione ha permesso di rispettare il vincolo di separazione tra struttura e logica senza introdurre la complessità di configurazione di un motore di template esterno o librerie esterne non necessarie per il dominio dell'applicazione.

7.2 Utilizzo del codice fiscale come identificatore univoco logico

Nel contesto specifico di un portale sanitario, l'identificazione certa del cittadino è un requisito fondamentale. L'utilizzo del Codice Fiscale come chiave logica per l'autenticazione e la memorizzazione in sessione, anziché l'adozione di un classico nome utente di fantasia, risponde a criteri di utilità reale e accessibilità cognitiva. L'utente non è costretto a memorizzare un'ulteriore credenziale astratta, sfruttando un identificatore univoco statale già in suo possesso. Ai fini della tutela della riservatezza dei dati, tale stringa viene gestita esclusivamente in

memoria lato server e mai veicolata in chiaro all'interno dei parametri URL o nelle richieste di tipo GET.

7.3 Punti critici e limitazioni dell'architettura attuale

Il principale punto di attenzione emerso in fase di collaudo riguarda la gestione della concorrenza degli slot di prenotazione. Qualora due utenti inviino simultaneamente una richiesta per il medesimo orario, la validazione server esegue una lettura che potrebbe risultare non aggiornata prima dell'inserimento effettivo. Sebbene l'occorrenza sia mitigata dalla rapidità di esecuzione di MariaDB, un'evoluzione robusta del sistema richiederebbe l'introduzione di transazioni SQL con livello di isolamento stringente o l'applicazione di un vincolo di unicità sulla combinazione dei campi data, ora e stato direttamente sulla struttura della tabella. Questa possibile problematica è mitigata in parte dal ruolo dell'admin che può cancellare un eventuale prenotazione in caso di sovrapposizione.

8 Test

Durante lo sviluppo di VitalPath, oltre al controllo visivo diretto nel browser, abbiamo affiancato una serie di verifiche automatiche e strumentate, mirate a intercettare errori di markup, criticità di accessibilità e problemi di prestazioni prima della consegna.

8.1 Validazione del markup

Il primo livello di controllo riguarda la correttezza sintattica del codice prodotto. Abbiamo utilizzato Total Validator per verificare l'HTML5 generato dinamicamente da PHP, individuando gli errori in maniera più efficace ed efficiente. Lo stesso lavoro è stato ripetuto con diversi validatori online per il CSS e per il markup XML degli elementi SVG inline, come il logo presente nell'header, poiché una sintassi scorretta in un file SVG può impedirne il rendering in alcuni browser senza restituire nessun errore visibile in console. Le pagine PHP sono state validate dopo il rendering lato server, in modo da controllare l'HTML effettivamente inviato al browser e non il template sorgente.

8.2 Estensioni Chrome per l'analisi di accessibilità

Per approfondire il controllo sull'accessibilità abbiamo affiancato ai validatori sintattici quattro estensioni del browser. Dtools - Web Developer Tools ha permesso di ispezionare rapidamente la struttura del DOM e la gerarchia degli heading. WAVE segnala a colpo d'occhio, tramite icone sovrapposte direttamente sulla pagina, errori strutturali, alert e funzionalità mancanti per le tecnologie assistive. WCAG Contrast Checker ha reso possibile controllare il rapporto

di contrasto sugli elementi effettivamente renderizzati, un riscontro pratico rispetto ai valori calcolati manualmente nella sezione precedente. Accessibility Insights for Web ha aggiunto una scansione automatica e una modalità di verifica guidata passo per passo, quest'ultima particolarmente utile per controllare l'ordine di tabulazione e la coerenza del focus visibile tra gli elementi interattivi. Silktide Accessibility Checker è stato lo strumento più utile tra i cinque, grazie all'ampiezza della copertura WCAG 2.2. A differenza degli altri strumenti, evidenzia i problemi direttamente sulla pagina analizzata e affianca a ogni segnalazione una spiegazione passo per passo su come correggerla, riducendo il tempo necessario per interpretare l'errore. Ha inoltre un simulatore di screen reader integrato, che ha permesso di verificare rapidamente come il contenuto della pagina viene annunciato. Abbiamo inoltre usato la possibilità di cambiare dispositivo per testare il nostro sito su diversi dispositivi con dimensioni dello schema differenti.

8.3 Audit con Lighthouse

Tramite il pannello sviluppatore di Chrome (F12) abbiamo eseguito più volte l'audit di Lighthouse, che restituisce un punteggio sintetico su quattro categorie, prestazioni, accessibilità, best practice e SEO. Lo strumento si è rivelato utile soprattutto nelle fasi finali dello sviluppo, quando le correzioni puntuali suggerite dai singoli controlli rischiavano di far perdere di vista il quadro generale della pagina.

9 Note conclusive

Lo sviluppo della piattaforma VitalPath ha permesso al gruppo di affrontare l'intero ciclo di vita di un'applicazione web accessibile, traducendo le specifiche teoriche del corso in scelte ingegneristiche concrete. Il lavoro ha evidenziato come sia possibile ottenere un'interfaccia moderna, fluida e inclusiva senza ricorrere a librerie esterne pesanti, affidandosi esclusivamente alla pulizia del codice semantico HTML5 e alle potenzialità dei layout CSS Grid e Flexbox. L'adozione di strumenti di virtualizzazione (Docker) si è rivelata determinante per il successo del lavoro di squadra, annullando i tempi morti legati al disallineamento degli ambienti di sviluppo locali. Il risultato finale è un prototipo solido, scalabile e pienamente rispondente ai criteri normativi di accessibilità visiva e motoria, che unisce la rigorosità della sfida tecnica a un obiettivo di forte utilità sociale.